

Current Recursion and Tensor-Network Execution in PYAMPLICOL

Purpose. This note summarizes the matrix-element generation strategy used by AMPLICOL and the corresponding PYAMPLICOL implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AMPLICOL emits Fortran code that numerically sweeps this current table. PYAMPLICOL builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, and compiles the resulting parametric scalar/vector expressions with Symbolica evaluators.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}_{t,\chi}^\alpha(I) = P^\alpha_\beta(t, p_I) \sum_{\substack{I=A \cup B \\ A \cap B = \emptyset}} \mathcal{V}^\beta_{\gamma\delta}(t_A, t_B \rightarrow t) \mathcal{J}_{t_A, \chi_A}^\gamma(A) \mathcal{J}_{t_B, \chi_B}^\delta(B). \quad (1)$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i \in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AMPLICOL's optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = (t, \chi, \text{bin}(I), \text{ext_cur}), \quad (2)$$

where `bin(I)` records the external subset and `ext_cur` is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AMPLICOL appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```
subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine
```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that PYAMPLICOL keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

PYAMPLICOL first builds an explicit Python representation of the same current table. For the leading-color milestone $q\bar{q} \rightarrow Z + ng$, the current identity is

$$K_{\text{PYAMPLICOL}} = (\text{pdg}, \text{labels}, \chi, \text{source_bits}). \quad (3)$$

The `source_bits` field is the direct analogue of AMPLICOL's `ext_cur`. Source currents are created for the allowed quark, antiquark, gluon, and massive-vector helicity states. Internal currents are then generated by increasing current size, with all interactions that feed the same key attached to one node.

```

for external helicity source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained helicity ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()

```

At runtime, PYAMPLICOL mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \text{current_rows}[s, i, c], \quad T_{s,v,c} \equiv \text{interaction_rows}[s, v, c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```

fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents] = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2 = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)

```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and hep_lib

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}^P(p_A, p_B) A^\mu B^\nu, \quad \text{or in code: } V(m("a"), m("b"), m("out")) * A(m("a")) * B(m("b")).$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library `hep_lib` is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```

library = model.build_tensor_library() # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

```

```

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())

```

The important point is that `hep_lib` stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spensio executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, PYAMPLICOL interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AMPLICOL's execution model.

If explicit color tensors are introduced in future full-color modes, color simplification belongs before Spensio execution:

```
raw ← idenso.simplify_color(raw),      then build TensorNetwork(raw, hep_lib).
```

For the current leading-color milestone, color is already reduced to scalar factors associated with the ordered amplitude.

4 Example: $d\bar{d} \rightarrow Zgg$

The user-level process is written as

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_Z) g(p_3) g(p_4).$$

For the leading-color current recursion, the colored chain is treated in the canonical order

$$\bar{d} - d - g_3 - g_4,$$

with the color-singlet Z inserted on the quark line. The relevant source currents are

$$\mathcal{J}_{\bar{q}}(1, h_1), \quad \mathcal{J}_q(2, h_2), \quad \epsilon^\mu(3, h_3), \quad \epsilon^\nu(4, h_4), \quad \epsilon_Z^\rho(Z, h_Z).$$

The gluon subcurrents include

$$\mathcal{J}_g^\alpha(3, 4; h_3, h_4) = P_g^\alpha{}_\beta(p_3 + p_4) \mathcal{V}_{3g}^{\beta\mu\nu}(p_3, p_4) \epsilon_\mu(3, h_3) \epsilon_\nu(4, h_4), \quad (4)$$

$$\mathcal{J}_T^A(3, 4; h_3, h_4) = \mathcal{V}_{ggT}^{A\mu\nu} \epsilon_\mu(3, h_3) \epsilon_\nu(4, h_4), \quad (5)$$

where $\mathcal{J}_T$ is the auxiliary tensor current used to reproduce the four-gluon structure in larger examples. The quark line then forms

$$\mathcal{J}_q(2, 3) = P_q(p_2 + p_3) \mathcal{V}_{qqq}(\mathcal{J}_q(2), \epsilon(3)), \quad (6)$$

$$\mathcal{J}_q(2, 3, 4) = P_q(p_2 + p_3 + p_4) [\mathcal{V}_{qqq}(\mathcal{J}_q(2), \mathcal{J}_q(3, 4)) + \mathcal{V}_{qqq}(\mathcal{J}_q(2, 3), \epsilon(4))], \quad (7)$$

and the Z can be inserted before, between, or after the ordered gluons:

$$\mathcal{J}_q(2, Z) = P_q(p_2 + p_Z) \mathcal{V}_{qZq}(\mathcal{J}_q(2), \epsilon_Z), \quad (8)$$

$$\mathcal{J}_q(2, 3, Z) = P_q(p_2 + p_3 + p_Z) [\mathcal{V}_{qZq}(\mathcal{J}_q(2, 3), \epsilon_Z) + \mathcal{V}_{qqq}(\mathcal{J}_q(2, Z), \epsilon(3))], \quad (9)$$

with analogous terms leading to $\mathcal{J}_q(2, 3, 4, Z)$. The helicity amplitude is finally an endpoint contraction,

$$\mathcal{M}(h_1, h_2, h_3, h_4, h_Z) = \mathcal{J}_q(2, 3, 4, Z; h_2, h_3, h_4, h_Z) \cdot \mathcal{J}_{\bar{q}}(1, h_1). \quad (10)$$

Only retained helicity ancestries survive the filter. The reported squared matrix element has the same structure as in AMPLICOL:

$$|\mathcal{M}|_{\text{LC}}^2 = \mathcal{N}_{\text{AC}} \sum_{h \in H_{\text{kept}}} w_h |\mathcal{M}_h|^2, \quad (11)$$

where $\mathcal{N}_{\text{AC}}$ contains the leading-color normalization, couplings, symmetry factors, and initial-state averaging conventions matched to the Fortran implementation.

5 Toward a Single Aliased Multi-Output Evaluator

The current milestone deliberately stays close to the generated Fortran execution model. Python owns the current table and performs the topological sweep at runtime. Each step of that sweep calls bounded Spensio/Symbolica evaluator blocks that fill interaction rows, derived-current rows, and finally the retained amplitude vector. This is robust and memory-safe: no full matrix-element expression is ever expanded, and each block has controlled rank, parameter count, and output count. The price is that every phase-space batch crosses the Python boundary several times:

$$C_{\text{old}} = C_{\text{source}} \xrightarrow{\text{Python loop}} E_1(C, p) \xrightarrow{\text{Python table update}} E_2(C, p) \cdots \rightarrow E_{\text{amp}}(C, p).$$

Here C denotes the current table and each E_k is a compiled evaluator for one layer or bounded block. The next natural step is to keep the same AMPLICOL-style DAG construction but move this orchestration from runtime to generation time.

In that alternative design, the Python graph builder would still enumerate the same source currents, internal currents, interactions, and endpoint amplitude records. However, instead of executing a current block numerically and writing the result back to `current_rows`, every component of every generated current would receive an opaque symbolic alias:

$$J(i, c) \longleftrightarrow \text{component } c \text{ of current } i.$$

The right-hand side of this alias is the Spensio-executed parametric tensor component for that current. It may contain external wavefunction parameters, momentum parameters, model constants, and aliases of parent current components. The final roots passed to Symbolica are the retained amplitude components, expressed only in terms of endpoint current aliases and external parameters:

$$(\mathcal{M}_{h_1}, \dots, \mathcal{M}_{h_N}) = R(J(i_1, c_1), J(i_2, c_2), \dots; p, \epsilon, u, \bar{u}).$$

The evaluator is then a single multi-output object for the full amplitude vector. Runtime evaluation becomes one parameter-pack operation and one compiled evaluator call, followed by the same helicity/color weighted $|\mathcal{M}_h|^2$ reduction.

```
aliases = {}
alias_tensors = {}

for current in topological_currents:
    tensor_expr = build_spensio_current_tensor(
        current,
        parent_alias_tensors=alias_tensors,
        hep_lib=hep_lib,
    )
    executed = TensorNetwork(tensor_expr, hep_lib).execute()

    components = {}
    for component, atom in executed.components():
        handle = J(current.id, component)
        aliases[handle] = atom
        components[component] = handle
    alias_tensors[current.id] = tensor_from_alias_components(components)

roots = [
    build_amplitude_component(amplitude, alias_tensors)
    for amplitude in retained_amplitudes
]

evaluator = AliasedExpression(roots, aliases).evaluator(params)
```

The important distinction from registering new tensors in `hep_lib` is that `hep_lib` remains the fixed dictionary of physics and representation objects: metrics, propagators, vertices, and parametric external tensors. Current aliases are not new model tensors. They are scalar Symbolica handles for already-executed tensor-network components. Spensio is still responsible for contracting Lorentz, spinor, and auxiliary-tensor indices in each current definition. Symbolica is then responsible for compiling the alias graph into machine code.

The alias mechanism matters. A naive Symbolica `functions` map is not the right representation for the full current pool, because such function definitions are inlined during evaluator construction and only later rediscovered by common-subexpression elimination. That may recreate the large monolithic expressions that the staged design avoids. The intended primitive is `AliasedAtom`, or a Python binding exposing the same alias-aware evaluator construction.

Aliases are designed to be opaque handles with a separate map from handle to body. This should let the evaluator builder see the DAG as a pool of reusable current components rather than as a fully expanded tree.

The generation pass for this backend would therefore have two invariants. First, the current graph must remain topologically ordered, so every alias body references only external parameters and aliases that have already been registered. Second, each alias handle must be an indeterminate accepted by the Symbolica evaluator builder, for example a tagged function atom $J(i, c)$ or a generated symbol J_i_c . The first form is more diagnostic because it keeps the current id and component visible in printed debug output. The second form may be useful if the Python binding exposes only symbol aliases initially. In either case, the alias map is acyclic:

$$J(i, c) = F_{i,c}(p, \epsilon, u, \bar{u}, \{J(j, d) \mid j \in \text{parents}(i)\}), \quad j < i.$$

Helicity sharing would be preserved exactly as in the present DAG evaluator. The source-current ancestry bits are still part of the current key, so two currents with the same AMPLICOL identity share one alias tensor, while currents with different helicity ancestry remain distinct. The retained amplitude roots are not constructed by looping over full helicity configurations independently; they are read from the same filtered current table. Thus the single-evaluator backend is not a return to a diagrammatic or per-helicity construction. It is the same shared current table, but serialized into an alias graph rather than into a sequence of runtime stage calls.

The implementation should be validated with explicit size diagnostics before large multiplicities are attempted. For every generated process, the artifact should record

$$n_{\text{alias}}, \quad n_{\text{root}}, \quad \sum_i \text{ops}(F_i), \quad \max_i \text{ops}(F_i), \quad \text{bytes}(\text{root}, \text{aliases}),$$

alongside the existing current, interaction, and amplitude counts. A small MRE should compare three objects for low multiplicity: the current staged compiled evaluator, the fully aliased evaluator, and Fortran AMPLICOL. The printed input to evaluator construction must still show a factorized root plus aliases; calling an operation that unfolds aliases should be reserved for debugging only. If evaluator construction starts from an unfolded expression, this backend has failed its main design requirement.

There is also a useful intermediate mode. Instead of putting every current in one global alias pool immediately, PYAMPLICOL can build one aliased evaluator per current-size layer, with aliases shared inside the layer and explicit current table outputs between layers. This would test the Python binding and alias semantics on smaller objects while still reducing the number of runtime calls. Once that is validated, adjacent layers can be fused until the final artifact contains the full amplitude vector. The final target remains one evaluator, but staged fusion gives a controlled path for diagnosing memory and compile time failures.

For $d\bar{d} \rightarrow Zgg$, the alias pool would start with source handles such as

$$J(q_2, h_2, c), \quad J(\bar{q}_1, h_1, c), \quad J(g_3, h_3, \mu), \quad J(g_4, h_4, \nu), \quad J(Z, h_Z, \rho).$$

The two-gluon current becomes, component by component,

$$J(g_{34}, \alpha) := P_g^\alpha{}_\beta (p_3 + p_4) \mathcal{V}_{3g}^{\beta\mu\nu}(p_3, p_4) J(g_3, \mu) J(g_4, \nu), \quad (12)$$

and the quark-line current with a Z insertion can reference both source and derived aliases:

$$J(q_{234Z}, c) := [P_q \mathcal{V}_{qqq}(J(q_2), J(g_{34})) + P_q \mathcal{V}_{qqq}(J(q_{23Z}), J(g_4)) + \cdots]_c. \quad (13)$$

The final retained roots are endpoint contractions such as

$$R_h = \sum_c J(q_{234Z}; h, c) J(\bar{q}_1; h, c), \quad (14)$$

one root for each retained helicity ancestry. A single evaluator would return all R_h values at once. The summed matrix element remains a lightweight post-processing operation unless the final weighted norm is also added as an extra root.

This direction has three expected advantages. First, runtime transfers should drop substantially because intermediate current tables no longer cross the Python boundary. Second, Symbolica can optimize across the complete retained amplitude vector rather than only within bounded stage blocks. Third, the resulting artifact more closely resembles a generated library: load one evaluator, pack parameters, evaluate all amplitudes. The main risks are generation memory, compile time, and alias semantics. The implementation must prove that alias definitions remain preserved through evaluator construction and are not expanded before optimization. It will likely require adding a small Python binding for Symbolica's alias-aware evaluator API before this backend can be tested fairly.

6 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, evaluator caching, and validation against AMPLICOL. It does not perform the hot algebraic recursion component by component. The hot path is a sequence of compiled evaluator calls over batches of phase-space points:

source wavefunctions $\longrightarrow$ stage evaluators $\longrightarrow$ amplitude/raw-sum evaluator.

Thus PYAMPLICOL keeps AMPLICOL’s essential optimization—shared currents across helicity parents and a topological current-table sweep—while replacing generated Fortran subroutines by Spenso/ Symbolica evaluator artifacts that can be saved, loaded, and benchmarked independently.